دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

Computer Architecture

Chapter 4
Performance

Afarghadan@aut.ac.ir

عظیم فرقدان

بهمن ۱۴۰۳

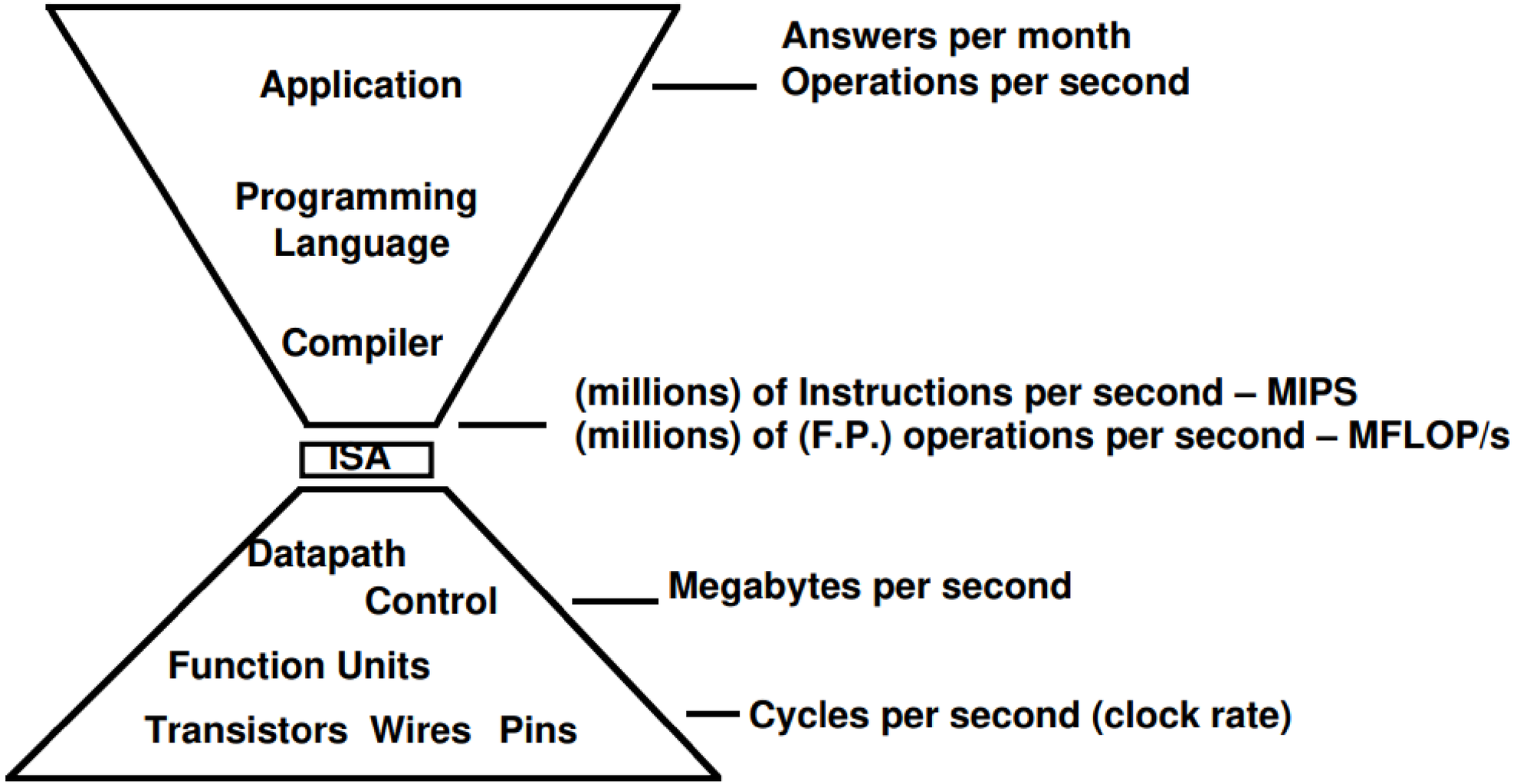
Lecture Overview

1. **Performance**
2. **CPU time**
3. **Benchmark**
4. **Performance Metrics**
5. **Speedup**
6. **Power**
7. **From Uniprocessors to Multiprocessors**

Defining Performance

- When trying to choose among different computers, performance is an important attribute.
- Accurately measuring and comparing different computers is critical to buyers and designers.
- When we say one computer has **better performance** than another, what do we mean?
- In most cases, we need different performance metrics as well as different sets of **applications** to **benchmark** different use cases.

Levels of Performance



Performance Measures

- **Response time (Execution time):**
  - The total time required for the computer to complete a task, including disk accesses, memory accesses, I/O activities, operating system overhead, CPU execution time,…

- **Throughput (Bandwidth):**
  - Number of tasks completed per unit time.

- To maximize performance, we want to minimize response time for some task:

$$Performance \propto \frac{1}{Execution\ Time}$$

Performance Measures

- **Relative Performance**
  - To compare the performance of two computers

$$\frac{Performance_A}{Performance_B} = \frac{Execution\ Time_B}{Execution\ Time_A}$$

- Example:
  - If computer A runs a program in 10 seconds and computer B runs the same program in 15 seconds, how much faster is A than B?

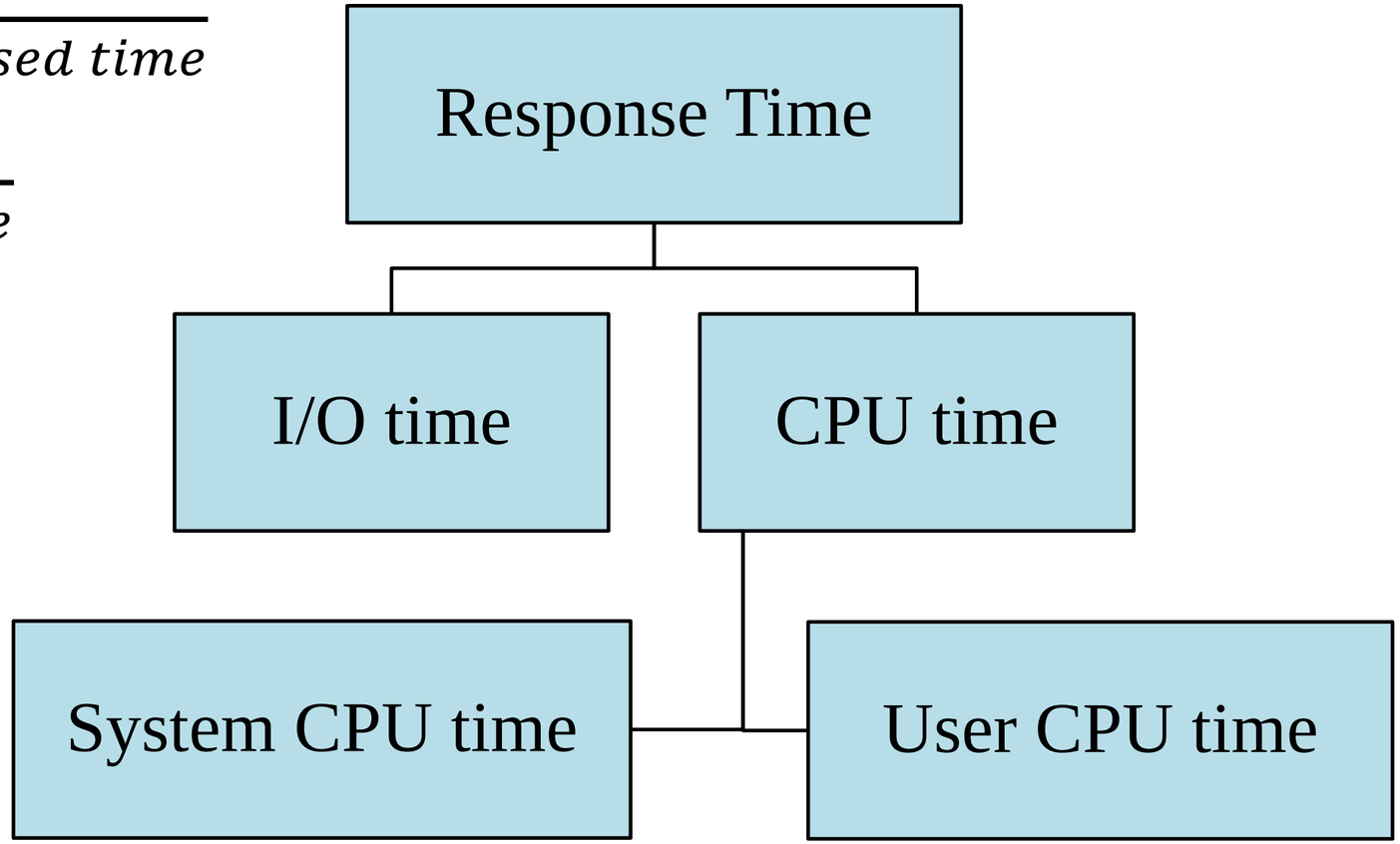
Program Execution Time

- **Time** is the measure of computer performance
- **wall clock time**, **response time**, **Execution time**, or **elapsed time.**
  - These terms mean the total time to complete a task, including disk accesses, memory accesses, input/output (I/O) activities, operating system overhead— everything.

- **CPU time:** Time spent processing the program
  - The I/O time and other programs' share of time is excluded
- CPU time consists of:
  - **User CPU time**: CPU time spent on the program
  - **System CPU time**: CPU time that is spent on OS doing tasks on behalf of program.

Response Time Components

- $Performance = \frac{1}{response\ time}$
- $System\ performance = \frac{1}{elapsed\ time}$
- $CPU\ performance = \frac{1}{CPU\ time}$



Understanding Program Performance

- Different applications are sensitive to **different aspects** of the performance of a computer system.
  - Server applications depend on I/O performance. Therefore, total elapsed time measured by a wall clock is the measurement of interest.
  - In some application environments, the user may care about throughput, response time, or a **complex combination** of the two (e.g., maximum throughput with a worst-case response time).
- To improve the performance of a program, one must have a **clear definition of what performance metric matters** and then proceed to **find performance bottlenecks** by measuring program execution and looking for the likely bottlenecks.

Clock Cycles

- Almost all computers are constructed using a clock that determines when events take place in the hardware.
- These discrete time intervals are called **clock cycles**, **ticks**, **clock ticks**, **clock periods**, **clocks**, **cycles**.
- The length of a clock is called **clock period**.
  - E.g., 250 picoseconds, or 250 ps
- **Clock rate (Frequency) = 1/Clock period**
  - E.g., 4 gigahertz, or 4 GHz

CPU Time

- **CPU Time** = number of Clock Cycles * Clock Cycle Time
  = number of Clock Cycles * T
  = number of Clock Cycles / F
  ⁕ F: CPU Frequency (Hz)

- The hardware designer can **improve performance** by reducing the number of clock cycles of a program or the length of the clock cycle.
- **Example**: Which one's faster? Computer A or Computer B?

Computer	Frequency	# Clocks
A	3.6 GHz	2000
B	2.6 GHz	1500

CPI

- **CPI**
  - Number of Clocks per Instruction
  - Average number of clock cycles per instruction for a program or program fragment.

- **Number of Overall Clock Cycles** = number of Instructions * *CPI*

- **Example**: Calculate CPU Time:

# of Instructions	4,000
CPI	3
Frequency	2.6 GHz

CPI

- If different instructions had different CPIs:
  - CPI = Average CPI in all Instructions

$$CPI = \frac{1}{n}\sum_{i=1}^{n} \#\ of\ Clk\ for\ Instr._i$$

N: classes of instr.

- CPU Time = IC * CPI * T
  = IC * CPI / F
- IC: the number of instructions executed by the program
  - Code Size or Lines of Code???

CPI - Example

- Assume a processor with instruction frequencies and costs
  - Integer ALU: 50%, 1 cycle
  - Load: 20%, 5 cycle
  - Store: 10%, 1 cycle
  - Branch: 20%, 2 cycle

- Which change would improve performance more?

  A. “Branch prediction” to reduce branch cost to 1 cycle?

  B. Faster data memory to reduce load cost to 3 cycles?

CPU Time

- CPU Time = **IC** * **CPI** * **T** = **IC** * **CPI** / **F**
- How to determine each parameter?
  - **IC**
    - Program (Algorithm, Language)
    - ISA
    - Compiler
  - **CPI**
    - uArch: The way processor is implemented
    - CPI also depends on the program
  - **Frequency (F)**
    - Technology and uArch

	instr count	CPI	clock rate
Program	X		
Compiler	X	(x)	
Instr. Set.	X	X	
Organization		X	X
Technology			X

CPU Time

HW / SW	Affects	How?
Algorithm	IC, CPI	The algorithm determines the number of source program instructions executed and hence the number of processor instructions executed. The algorithm may also affect the CPI, by favoring slower or faster instructions. For example, if the algorithm uses more divides, it will tend to have a higher CPI.
Programming Language	IC, CPI	The programming language certainly affects the instruction count, since statements in the language are translated to processor instructions, which determine instruction count. The language may also affect the CPI because of its features; for example, a language with heavy support for data abstraction (e.g., Java) will require indirect calls, which will use higher CPI instructions.
Compiler	IC, CPI	The efficiency of the compiler affects both the instruction count and average cycles per instruction, since the compiler determines the translation of the source language instructions into computer instructions. The compiler's role can be very complex and affect the CPI in varied ways.
Instruction Set Architecture	IC, F, CPI	The instruction set architecture affects all three aspects of CPU performance, since it affects the instructions needed for a function, the cost in cycles of each instruction, and the overall clock rate of the processor.

Benchmark

- Consider Two CPUs
  - On program A, CPU#1 runs **faster** than CPU#2
  - On program B, CPU#1 runs **slower** than CPU#2

- So...
  - Which **CPU** is faster?
  - Which **program** should we choose to compare CPUs?

- **Benchmarks**: a **set of programs** used to compare the performance of processors.

Benchmark

- Evaluate difference in:
  - Different systems
  - Changes to a single system
- Provide a target
  - Benchmarks should represent **large class of important programs**
  - Improving benchmark performance should **help many programs**
- Benchmarks shape a field
  - Good benchmarks **help progress** by being good targets for development
  - Bad benchmarks **hurt progress**

Performance Metrics

- **MIPS**
  - A measurement of program execution speed based on the number of millions of instructions.
  - **Drawbacks**
    - Capabilities of instructions not taken into account
      - Comparing two different ISAs isn’t fair
    - Not a realistic metric EVEN on the same CPUs
    - If MIPS(A) > MIPS(B) can we conclude that performance(A) > performance(B)?

Further on MIPS

$$\text{IPC} = \frac{1}{CPI}$$

$$\text{MIPS} = \frac{\#\ Instructions}{10^6 \times \#\ Seconds}$$

$$\text{MIPS} = \frac{\#\ Instructions}{10^6 \times \#Clocks \times \frac{1}{f}} = \frac{IPC}{10^6 \times \frac{1}{f}} = IPC \times f \times 10^{-6} = \frac{1}{CPI} \times f \times 10^{-6}$$

$$Execution\ Time \simeq \frac{n}{IPC} \times \frac{1}{f} = \frac{n}{MIPS} \times f \times 10^{-6} \times \frac{1}{f} = \frac{n \times 10^{-6}}{MIPS}$$

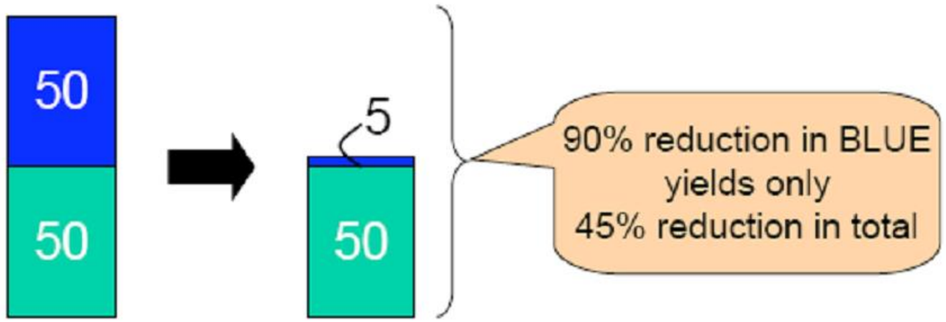
Performance Metrics

- **MFLOPS**
  - Million Floating Point Operations per Second
  - Popular in scientific computing

- **Drawbacks**
  - Ignores every other instruction (Load/Store)
  - Not all floating point operations have common format

Speedup

- **Speedup** = $\text{ExecTime}_{\text{origin}}$ / $\text{ExecTime}_{\text{Improved}}$
- Obvious but common **mistake**:
  - Expecting the improvement of one aspect of a computer to increase overall performance by an amount **proportional** to the size of the improvement.
- **Amdahl's Law**: performance enhancement possible with a given improvement is limited by the amount that the improved feature is used.



Amdahl's Law

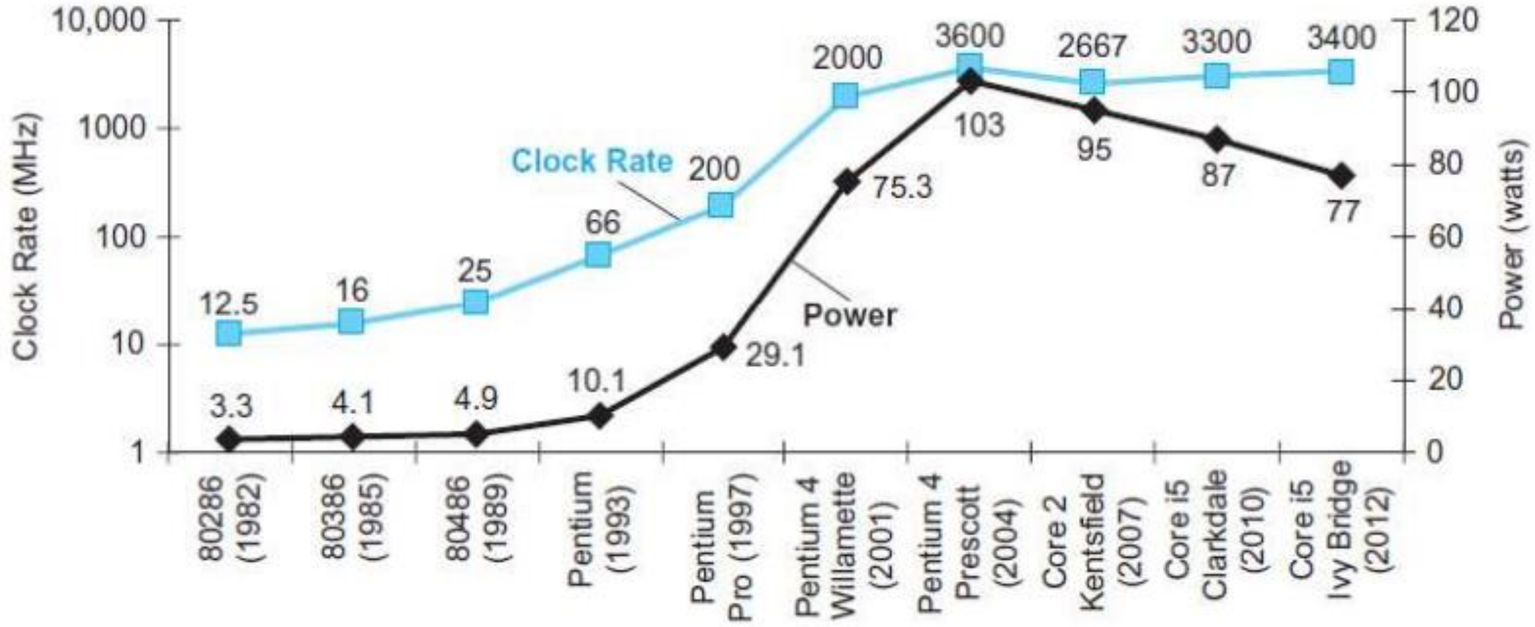
Execution time after improvement =

Execution time unaffected + $\frac{\textbf{Execution time affected by improvement}}{\textbf{amount of improvement}}$

$$T_{exe,new} = T_{exe,old} \times [(1 - f_{enhanced}) + (f_{enhanced}/speedup_{enhanced})]$$

$$Speedup_{overall} = \frac{1}{(1 - f_{enhanced}) + (f_{enhanced}/speedup_{enhanced})}$$

- **Power Wall**
  - For environmental reasons, we cannot consume power more than a threshold. Otherwise, we're unable to maintain its overall thermal stability
  - **Frequency ∝ power**



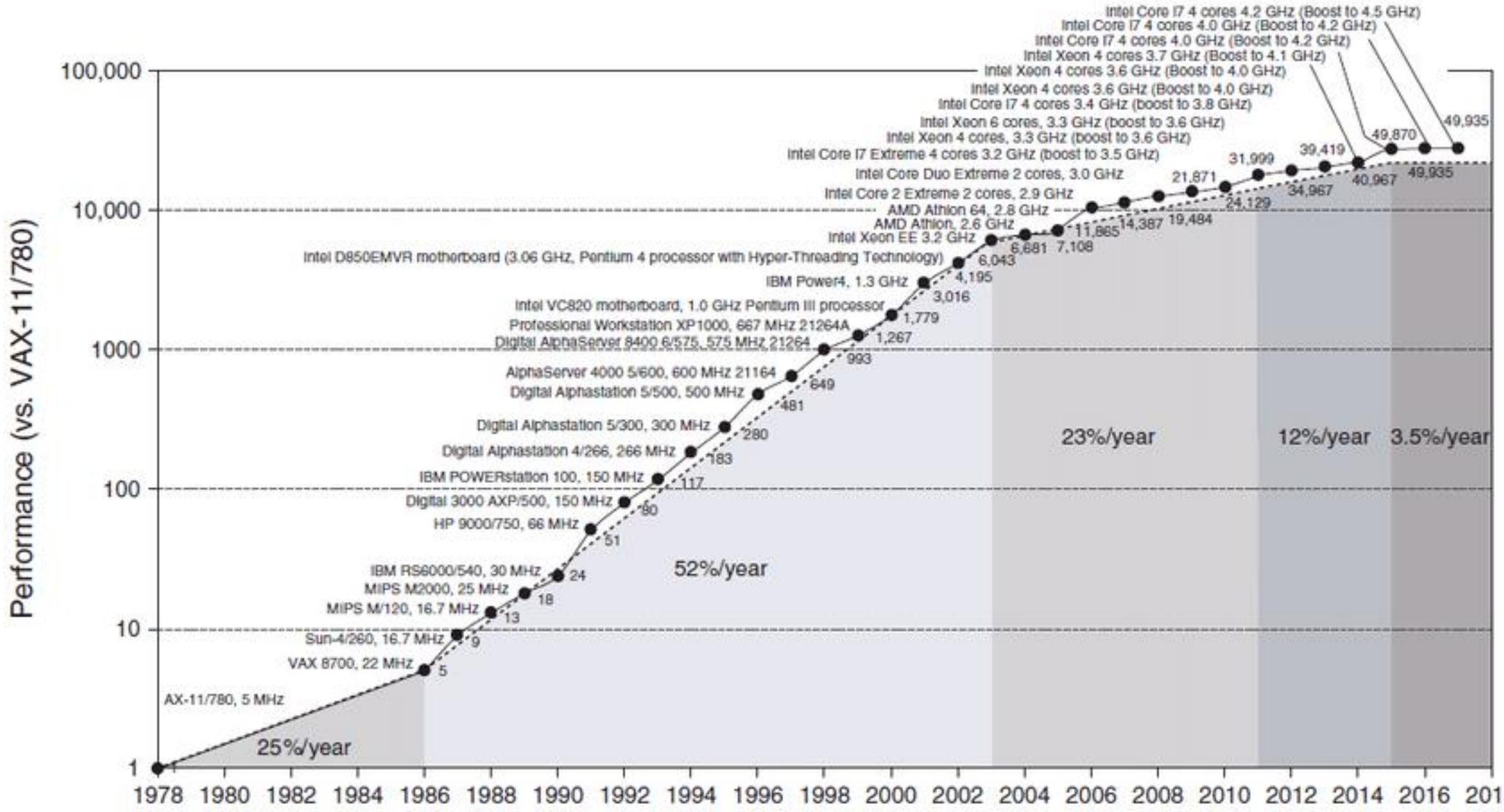
Power

- Energy
  - Can only go from 0 to 1 or from 1 to 0
  - Dynamic Energy: Consumed energy in order for a state transition (0 to 1 or 1 to 0)
  - Static Energy: Consumed energy when we don’t have a state transition (Idle) due to leakage current in transistors even when they are off.

$$Energy \propto 1/2 \times Capacitive\ load \times Voltage^2$$

$$Power \propto 1/2 \times Capacitive load \times Voltage^2 \times Frequency switched$$

From Uniprocessors to Multiprocessors



Thanks